

```

# BLOB Principle - Protocole F CORRIGE - Mesures hierarchiques
# Version 2 : correction du bug de comptage des tetraedres
#
# Bug identifie dans F v1 :
#   La fonction _count_tetra_through_edge n etait pas appelee correctement
#   lors des swaps. Le delta tetraedre etait toujours 0.
#
# Correction v2 :
#   Le calcul de delta_tetra utilise un algorithme robuste qui compte
#   les tetraedres detruits (impliquant 1 arete retiree) et les tetraedres
#   crees (impliquant la nouvelle arete) sur la base de l etat ACTUEL
#   du graphe, sans dependre des swaps precedents.

import sys
import os

# =====
# AUTO-TEST CORRUPTION
# =====

def auto_check():
    print("=" * 70)
    print("  AUTO-TEST CORRUPTION CARACTERES")
    print("=" * 70)
    try:
        if hasattr(sys, 'argv') and len(sys.argv) > 0 and os.path.exists(sys.argv
            source_file = sys.argv[0]
        else:
            source_file = __file__ if '__file__' in globals() else None
        if source_file and os.path.exists(source_file):
            with open(source_file, 'r', encoding='utf-8') as f:
                content = f.read()
            dangerous = {
                chr(8216): "U+2018", chr(8217): "U+2019",
                chr(8220): "U+201C", chr(8221): "U+201D",
                chr(8211): "U+2013", chr(8212): "U+2014",
            }
            found = [(name, content.count(c)) for c, name in dangerous.items() if
                if found:
                    print("ERREUR caracteres corrompus :", found)
                    return False
            star = chr(42)
            bad_init = star + star + "init" + star + star
            if content.count(bad_init) > 1:
                print("ERREUR : motif corrompu init")

```

```

        return False
    triple_back = chr(96) * 3
    if content.count(triple_back) > 1:
        print("ERREUR : Markdown")
        return False
    print("OK : pas de corruption")
else:
    print("Note : auto-test partiel")
except Exception as e:
    print("Note : " + str(e))
print("=" * 70)
return True

if not auto_check():
    sys.exit(1)

# =====
# IMPORTS
# =====
import numpy as np
import time
import json
import gc
from collections import defaultdict, Counter

# =====
# Parametres canoniques
# =====
KAPPA_0 = 8
BETA = 0.30
TAU = 0.50

OUTPUT_DIR = '/content/blob_F2_results'
os.makedirs(OUTPUT_DIR, exist_ok=True)

# =====
# Classe BlobGraph etendue avec mesures hierarchiques CORRIGÉES
# =====
class BlobGraphHier:
    def __init__(self, N, seed, is_null=False):
        self.N = N
        self.is_null = is_null
        self.rng = np.random.default_rng(seed)
        self.E_target = int(N * KAPPA_0 / 2)

```

```

self.adj = [set() for _ in range(N)]
self.edge_list = []
self._next_edge_id = 0
self.edge_birth = {}
self.edge_death = {}
self.edge_endpoints = {}
self.edge_engagement_profile = {}
self.edge_max_engagement = {}
self.edge_position_to_id = {}

self._init_graph()
self.deg = np.array([len(self.adj[i]) for i in range(N)], dtype=np.int32)
self.n_tri = self._count_triangles()
self.n_tetra = self._count_tetrahedra()
self.S_curr = self._compute_S(self.n_tri)
self.transitions = Counter()

def _new_edge_id(self):
    eid = self._next_edge_id
    self._next_edge_id += 1
    return eid

def _add_edge_tracked(self, u, v, step):
    u, v = min(u, v), max(u, v)
    eid = self._new_edge_id()
    self.adj[u].add(v)
    self.adj[v].add(u)
    idx = len(self.edge_list)
    self.edge_list.append((u, v))
    self.edge_position_to_id[idx] = eid
    self.edge_birth[eid] = step
    self.edge_endpoints[eid] = (u, v)
    self.edge_engagement_profile[eid] = []
    self.edge_max_engagement[eid] = 0
    return eid

def _remove_edge_tracked(self, idx, step):
    u, v = self.edge_list[idx]
    eid = self.edge_position_to_id[idx]
    self.adj[u].discard(v)
    self.adj[v].discard(u)
    self.edge_death[eid] = step

    last_idx = len(self.edge_list) - 1
    if idx != last_idx:
        last_edge = self.edge_list[last_idx]

```

```

        last_eid = self.edge_position_to_id[last_idx]
        self.edge_list[idx] = last_edge
        self.edge_position_to_id[idx] = last_eid
    self.edge_list.pop()
    del self.edge_position_to_id[last_idx]

def _init_graph(self):
    for i in range(self.N):
        u = i
        v = (i + 1) % self.N
        if v not in self.adj[u]:
            self._add_edge_tracked(u, v, 0)
    attempts = 0
    max_a = self.E_target * 10
    while len(self.edge_list) < self.E_target and attempts < max_a:
        u = int(self.rng.integers(self.N))
        v = int(self.rng.integers(self.N))
        attempts += 1
        if u != v and v not in self.adj[u]:
            self._add_edge_tracked(u, v, 0)

def _count_triangles(self):
    return sum(len(self.adj[u] & self.adj[v]) for u, v in self.edge_list) //

def _count_tetrahedra(self):
    """
    Comptage exhaustif des 4-cliques.
    Cher en  $O(|E| * \deg^2)$  mais correct.
    """
    n_tetra = 0
    seen = set()
    for u, v in self.edge_list:
        common = self.adj[u] & self.adj[v]
        if len(common) < 2:
            continue
        common_list = sorted(common)
        for i, w in enumerate(common_list):
            for x in common_list[i+1:]:
                if x in self.adj[w]:
                    key = tuple(sorted([u, v, w, x]))
                    if key not in seen:
                        seen.add(key)
                        n_tetra += 1
    return n_tetra

def _tetra_count_through_edge(self, u, v):
    """

```

```

Nombre de tetraedres impliquant 1 arete (u,v) DANS L ETAT COURANT.
= nombre de paires (w,x) avec w,x dans N(u) inter N(v) et w-x edge.
"""

common = self.adj[u] & self.adj[v]
if len(common) < 2:
    return 0
common_list = list(common)
count = 0
for i, w in enumerate(common_list):
    adj_w = self.adj[w]
    for x in common_list[i+1:]:
        if x in adj_w:
            count += 1
return count

def _compute_S(self, n_tri_val):
    mu = float(self.deg.mean())
    D = float(self.deg.std()) / mu if mu > 0 else 0.0
    return np.log(1 + n_tri_val) - BETA * D

def step_blob(self, current_step):
    """
    Pas Blob avec calcul correct de delta_tri ET delta_tetra.

    Strategie corrige :
    1. Choisir arete a retirer (u_rm, v_rm) et arete a ajouter (u_add, v_add)
    2. Calculer dt_rm = triangles a retirer = |N(u_rm) inter N(v_rm)|
    3. Calculer dtetra_rm = tetraedres impliquant (u_rm, v_rm)
    4. Retirer (u_rm, v_rm)
    5. Calculer dt_add = triangles que (u_add, v_add) creerait = |N(u_add) in
    6. Ajouter (u_add, v_add) temporairement
    7. Calculer dtetra_add = tetraedres impliquant (u_add, v_add) maintenant
    8. delta_tri = dt_add - dt_rm
    9. delta_tetra = dtetra_add - dtetra_rm
    10. Acceptation Metropolis sur new_S
    11. Si rejete : annuler les modifications de etat
    """
    idx = self.rng.integers(len(self.edge_list))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False, None
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)
    if (u_add, v_add) == (u_rm, v_rm):
        return False, None

```

```

# Etape 1 : compter ce qui sera detruit en retirant (u_rm, v_rm)
dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
dtetra_rm = self._tetra_count_through_edge(u_rm, v_rm)

# Etape 2 : retirer temporairement (u_rm, v_rm)
self.adj[u_rm].discard(v_rm)
self.adj[v_rm].discard(u_rm)
self.deg[u_rm] -= 1
self.deg[v_rm] -= 1

# Etape 3 : compter ce qui sera cree en ajoutant (u_add, v_add)
# ATTENTION : on calcule cela APRES avoir retire (u_rm, v_rm)
# car les voisinages ont ete modifies
dt_add = len(self.adj[u_add] & self.adj[v_add])

# Pour les tetraedres : il faut ajouter temporairement l arete et compter
self.adj[u_add].add(v_add)
self.adj[v_add].add(u_add)
self.deg[u_add] += 1
self.deg[v_add] += 1
dtetra_add = self._tetra_count_through_edge(u_add, v_add)

# Calcul de la nouvelle viabilite
new_n_tri = self.n_tri - dt_rm + dt_add
if new_n_tri < 0:
    # Annuler tout
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False, None

new_S = self._compute_S(new_n_tri)
log_alpha = self.N * (new_S - self.S_curr) / TAU
if log_alpha >= 0:
    accepted = True
else:
    accepted = self.rng.random() < np.exp(max(log_alpha, -500))

if not accepted:
    # Annuler : retirer u_add, v_add et restaurer u_rm, v_rm
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)

```

```

        self.deg[u_add] -= 1
        self.deg[v_add] -= 1
        self.adj[u_rm].add(v_rm)
        self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1
        self.deg[v_rm] += 1
        return False, None

# ACCEPTATION : on a deja retire (u_rm, v_rm) et ajoute (u_add, v_add)
# dans les structures adj. Il reste a mettre a jour edge_list et tracking
# Restauration temporaire pour appeler les fonctions tracking proprement
self.adj[u_add].discard(v_add)
self.adj[v_add].discard(u_add)
self.deg[u_add] -= 1
self.deg[v_add] -= 1
self.adj[u_rm].add(v_rm)
self.adj[v_rm].add(u_rm)
self.deg[u_rm] += 1
self.deg[v_rm] += 1

# Maintenant faire la transition propre avec tracking
self._remove_edge_tracked(idx, current_step)
self.deg[u_rm] -= 1
self.deg[v_rm] -= 1
self._add_edge_tracked(u_add, v_add, current_step)
self.deg[u_add] += 1
self.deg[v_add] += 1

# Mise a jour des compteurs
delta_tri = dt_add - dt_rm
delta_tetra = dtetra_add - dtetra_rm
self.n_tri = new_n_tri
self.n_tetra += delta_tetra
self.S_curr = new_S

# Classification de la transition
if delta_tri == 0 and delta_tetra == 0:
    ttype = 'alpha'
elif delta_tri > 0 and delta_tetra == 0:
    ttype = 'beta'
elif delta_tri > 0 and delta_tetra > 0:
    ttype = 'gamma'
elif delta_tri < 0 and delta_tetra < 0:
    ttype = 'delta'
elif delta_tri == 0 and delta_tetra < 0:
    ttype = 'epsilon'
elif delta_tri < 0 and delta_tetra == 0:

```

```

        ttype = 'zeta'
    elif delta_tri == 0 and delta_tetra > 0:
        ttype = 'theta'
    else:
        ttype = 'eta'
    self.transitions[ttype] += 1

    return True, ttype

def step_null(self, current_step):
    idx = self.rng.integers(len(self.edge_list))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False, None
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)
    if (u_add, v_add) == (u_rm, v_rm):
        return False, None

    dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
    dtetra_rm = self._tetra_count_through_edge(u_rm, v_rm)

    self.adj[u_rm].discard(v_rm)
    self.adj[v_rm].discard(u_rm)
    self.deg[u_rm] -= 1
    self.deg[v_rm] -= 1

    dt_add = len(self.adj[u_add] & self.adj[v_add])
    new_n_tri = self.n_tri - dt_rm + dt_add

    if new_n_tri < 0:
        self.adj[u_rm].add(v_rm)
        self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1
        self.deg[v_rm] += 1
        return False, None

    # Acceptation auto en null
    self.adj[u_add].add(v_add)
    self.adj[v_add].add(u_add)
    self.deg[u_add] += 1
    self.deg[v_add] += 1
    dtetra_add = self._tetra_count_through_edge(u_add, v_add)

    # Annuler pour faire le tracking propre
    self.adj[u_add].discard(v_add)

```

```

        self.adj[v_add].discard(u_add)
        self.deg[u_add] -= 1
        self.deg[v_add] -= 1
        self.adj[u_rm].add(v_rm)
        self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1
        self.deg[v_rm] += 1

        self._remove_edge_tracked(idx, current_step)
        self.deg[u_rm] -= 1
        self.deg[v_rm] -= 1
        self._add_edge_tracked(u_add, v_add, current_step)
        self.deg[u_add] += 1
        self.deg[v_add] += 1

        self.n_tri = new_n_tri
        self.n_tetra += (dtetra_add - dtetra_rm)
        self.S_curr = self._compute_S(new_n_tri)

    return True, None

def update_engagement_profiles(self, current_step):
    for idx, (u, v) in enumerate(self.edge_list):
        eid = self.edge_position_to_id[idx]
        n_common = len(self.adj[u] & self.adj[v])
        self.edge_engagement_profile[eid].append((current_step, n_common))
        if n_common > self.edge_max_engagement[eid]:
            self.edge_max_engagement[eid] = n_common

# =====
# RUN
# =====
def run_F2(N, seed, total_steps, is_null, label='F2'):
    t_start = time.time()
    type_label = 'NULL' if is_null else 'BLOB'
    log_id = ' [' + label + ' ' + type_label + ' N=' + str(N) + ' s=' + str(seed)
    print(log_id + ' Init ' + str(total_steps) + ' steps...', flush=True)

    sim = BlobGraphHier(N=N, seed=seed, is_null=is_null)
    step_func = sim.step_null if is_null else sim.step_blob

    print(log_id + ' Init OK |E|=' + str(len(sim.edge_list))
          + ' n_tri=' + str(sim.n_tri)
          + ' n_tetra=' + str(sim.n_tetra)
          + ' init_t=' + '{:.0f}'.format(time.time() - t_start) + 's', flush=True)

```

```

n_samples = 100
sample_steps = sorted(set([
    max(1, int(np.exp(np.log(total_steps) * i / n_samples)))
    for i in range(1, n_samples + 1)
]))

engagement_sample_count = 20
engagement_steps = sorted(set([
    max(1, int(total_steps * i / engagement_sample_count))
    for i in range(1, engagement_sample_count + 1)
]))

trajectory = []
next_traj_idx = 0
next_eng_idx = 0
accepts = 0

for i in range(total_steps):
    accepted, ttype = step_func(i + 1)
    if accepted:
        accepts += 1
    if next_traj_idx < len(sample_steps) and (i + 1) >= sample_steps[next_traj_idx]:
        mu_deg = float(sim.deg.mean())
        D_curr = float(sim.deg.std()) / mu_deg if mu_deg > 0 else 0
        trajectory.append({
            'step': i + 1,
            'n_tri': sim.n_tri,
            'n_tetra': sim.n_tetra,
            'S': sim.S_curr,
            'viab_total': sim.S_curr * sim.N,
            'D': D_curr,
            'acc_cumul': accepts / (i + 1),
            'transitions_snapshot': dict(sim.transitions)
        })
        next_traj_idx += 1
    if next_traj_idx % 20 == 0:
        print('    step ' + str(i+1) + '/' + str(total_steps)
              + ' n_tri=' + str(sim.n_tri)
              + ' n_tetra=' + str(sim.n_tetra)
              + ' acc=' + '{:.3f}'.format(accepts/(i+1))
              + ' t=' + '{:.0f}'.format(time.time()-t_start) + 's', flush=True)
    if next_eng_idx < len(engagement_steps) and (i + 1) >= engagement_steps[next_eng_idx]:
        sim.update_engagement_profiles(i + 1)
        next_eng_idx += 1

total_time = time.time() - t_start
final_acc = accepts / total_steps if total_steps > 0 else 0

```

```

# Verification de coherence : recompter les tetraedres
n_tetra_recount = sim._count_tetrahedra()
print(log_id + ' n_tetra incremental=' + str(sim.n_tetra)
      + ' n_tetra recount=' + str(n_tetra_recount)
      + ' (coherence ' + ('OK' if sim.n_tetra == n_tetra_recount else 'KO!'))

# Conservation simplexiale
fl_simp = (sim.deg.sum() == 2 * sim.E_target)

# Drift viabilite
if len(trajectory) >= 4:
    viab_first_q = np.mean([t['viab_total'] for t in trajectory[:len(trajecto
    viab_last_q = np.mean([t['viab_total'] for t in trajectory[3*len(trajecto
    viab_drift = (viab_last_q - viab_first_q) / abs(viab_first_q) if viab_fir
else:
    viab_drift = 0

# Lifetimes
edge_lifetimes_dead = []
edge_max_engagement_dead = []
for eid, death in sim.edge_death.items():
    birth = sim.edge_birth.get(eid, 0)
    edge_lifetimes_dead.append(death - birth)
    edge_max_engagement_dead.append(sim.edge_max_engagement.get(eid, 0))

by_cat = {'isolee': [], 'engagee_tri': [], 'engagee_tetra_plus': []}
for lt, max_eng in zip(edge_lifetimes_dead, edge_max_engagement_dead):
    if max_eng == 0:
        by_cat['isolee'].append(lt)
    elif max_eng == 1:
        by_cat['engagee_tri'].append(lt)
    else:
        by_cat['engagee_tetra_plus'].append(lt)

lifetime_by_cat = {cat: float(np.mean(lst)) if lst else None for cat, lst in
counts_by_cat = {cat: len(lst) for cat, lst in by_cat.items()}

transitions_final = dict(sim.transitions)
total_t = sum(transitions_final.values())
ascensions = (transitions_final.get('beta', 0)
              + transitions_final.get('gamma', 0)
              + transitions_final.get('theta', 0))
regressions = (transitions_final.get('delta', 0)
               + transitions_final.get('epsilon', 0)
               + transitions_final.get('zeta', 0))
asy = (ascensions - regressions) / (ascensions + regressions) if (ascensions

```

```

save_path = OUTPUT_DIR + '/F2_' + label + '_' + type_label + '_N' + str(N) +
out = {
    'config': {'N': N, 'seed': seed, 'type': type_label, 'total_steps': total
    'final': {
        'n_tri_final': int(sim.n_tri),
        'n_tetra_final_incremental': int(sim.n_tetra),
        'n_tetra_recount_check': int(n_tetra_recount),
        'coherence': bool(sim.n_tetra == n_tetra_recount),
        'S_final': float(sim.S_curr),
        'acc_global': float(final_acc),
        'time_sec': float(total_time)
    },
    'F1_conservation': {
        'simplexial_conserved': bool(f1_simp),
        'viab_drift_relative': float(viab_drift)
    },
    'F2_aretes': {
        'edges_dead_total': len(sim.edge_death),
        'edges_alive_at_end': len(sim.edge_list),
        'lifetime_by_category_mean': lifetime_by_cat,
        'counts_by_category': counts_by_cat
    },
    'F3_transitions': {
        'counts': transitions_final,
        'total': total_t,
        'ascensions': ascensions,
        'regressions': regressions,
        'asymmetry_ratio': float(asy)
    },
    'trajectory': trajectory
}
try:
    with open(save_path, 'w') as f:
        json.dump(out, f)
except Exception as e:
    print(log_id + ' WARN save: ' + str(e))

print(log_id + ' DONE n_tri=' + str(sim.n_tri)
      + ' n_tetra=' + str(sim.n_tetra)
      + ' t=' + '{:.0f}'.format(total_time) + 's', flush=True)
if not is_null:
    print('    F1 simplex=' + str(f1_simp) + ' viab_drift=' + '{:+.3%}'.forma
    print('    F2 lifetimes:', flush=True)
    for cat, v in lifetime_by_cat.items():
        cnt = counts_by_cat[cat]
        print('        ' + cat + ' : '

```

```

        + ((' {:.0f}'.format(v)) if v else 'N/A')
        + ' pas (n=' + str(cnt) + ')', flush=True)
print('      F3 transitions :', flush=True)
for ttype in ['alpha', 'beta', 'gamma', 'delta', 'epsilon', 'zeta', 'thet
    cnt = transitions_final.get(ttype, 0)
    pct = 100 * cnt / total_t if total_t > 0 else 0
    label_t = {
        'alpha': 'reorganisation', 'beta': 'ascension Tri',
        'gamma': 'ascension Tetra (Tri+ Tetra+)', 'delta': 'regression co
        'epsilon': 'regression Tetra seule', 'zeta': 'regression Tri seul
        'theta': 'creation Tetra sans Tri', 'eta': 'mixte'
    }.get(ttype, ttype)
    print('      ' + ttype + ' (' + label_t + ') : ' + str(cnt)
        + ' ( {:.2f}% )'.format(pct), flush=True)
print('      F3 asymetrie=' + '{:+.3f}'.format(asy), flush=True)

del sim
gc.collect()
return out

# =====
# MAIN
# =====
print('')
print('=' * 75)
print('  PROTOCOLE F v2 (CORRIGE) : Mesures hierarchiques')
print('=' * 75)
print('  Version 2 : bug tetraedres corrige')
print('  Verification de coherence : recount final apres simulation')

N_FIXED = 50000
TOTAL_STEPS = 30 * N_FIXED

RUNS = [
    {'N': N_FIXED, 'seed': 0, 'is_null': False, 'label': 'F2.30N'},
    {'N': N_FIXED, 'seed': 1, 'is_null': False, 'label': 'F2.30N'},
    {'N': N_FIXED, 'seed': 100, 'is_null': True, 'label': 'F2.30N'},
]

print('')
print('  N=' + str(N_FIXED) + ', steps=' + str(TOTAL_STEPS) + ' (30N)')
print('  Total runs : ' + str(len(RUNS)))
print('  Duree estimee : 35-60 min')
print('=' * 75)

t_global = time.time()

```

```

all_results = []

for run_cfg in RUNS:
    print('')
    try:
        r = run_F2(run_cfg['N'], run_cfg['seed'], TOTAL_STEPS,
                    is_null=run_cfg['is_null'], label=run_cfg['label'])
        all_results.append(r)
    except Exception as e:
        print('  ERREUR : ' + str(e))
        import traceback
        traceback.print_exc()

global_time = time.time() - t_global

print('')
print('=' * 75)
print('  RAPPORT F v2')
print('=' * 75)
print('  Temps total : ' + '{:.1f}'.format(global_time / 60) + ' min')

print('')
print('  Verification coherence comptage tetraedres :')
for r in all_results:
    cfg = r['config']
    inc = r['final']['n_tetra_final_incremental']
    rec = r['final']['n_tetra_recount_check']
    coh = r['final']['coherence']
    print('    ' + cfg['type'] + ' s=' + str(cfg['seed'])
          + ' : incremental=' + str(inc)
          + ' recount=' + str(rec)
          + ' coherence=' + ('OK' if coh else 'KO!'))

blob_results = [r for r in all_results if r['config']['type'] == 'BLOB']
null_results = [r for r in all_results if r['config']['type'] == 'NULL']

print('')
print('  --- F1 Conservation ---')
for r in all_results:
    cfg = r['config']
    print('    ' + cfg['type'] + ' s=' + str(cfg['seed'])
          + ' : simplex=' + str(r['F1_conservation']['simplexial_conserved'])
          + ' viab_drift=' + '{:+.3%}'.format(r['F1_conservation']['viab_drift_re

print('')
print('  --- F2 Lifetimes par categorie (Blob) ---')
for r in blob_results:

```

```

cfg = r['config']
print(' Seed ' + str(cfg['seed']) + ' :')
for cat, v in r['F2_aretes']['lifetime_by_category_mean'].items():
    cnt = r['F2_aretes']['counts_by_category'][cat]
    print('      ' + cat + ' : '
          + ('{: .0f}'.format(v) if v else 'N/A')
          + ' pas (' + str(cnt) + ' aretes)')

# Verdict stabilisation
if blob_results:
    iso_lts = [r['F2_aretes']['lifetime_by_category_mean']['isolee']
               for r in blob_results
               if r['F2_aretes']['lifetime_by_category_mean']['isolee']]
    eng_lts = [r['F2_aretes']['lifetime_by_category_mean']['engagee_tri']
               for r in blob_results
               if r['F2_aretes']['lifetime_by_category_mean']['engagee_tri']]
    tetra_lts = [r['F2_aretes']['lifetime_by_category_mean']['engagee_tetra_plus']
                 for r in blob_results
                 if r['F2_aretes']['lifetime_by_category_mean']['engagee_tetra_pl
if iso_lts and eng_lts:
    ratio_eng = np.mean(eng_lts) / np.mean(iso_lts)
    print('')
    print(' Ratio lifetime engagee_tri / isolee : {:.2f}x'.format(ratio_eng))
if iso_lts and tetra_lts:
    ratio_tetra = np.mean(tetra_lts) / np.mean(iso_lts)
    print(' Ratio lifetime engagee_tetra+ / isolee : {:.2f}x'.format(ratio_t

print('')
print(' --- F3 Transitions (Blob) ---')
for r in blob_results:
    cfg = r['config']
    print(' Seed ' + str(cfg['seed']) + ' :')
    counts = r['F3_transitions']['counts']
    total_t = r['F3_transitions']['total']
    asy = r['F3_transitions']['asymmetry_ratio']
    for ttype in ['alpha', 'beta', 'gamma', 'delta', 'epsilon', 'zeta', 'theta',
                  cnt = counts.get(ttype, 0)
                  pct = 100 * cnt / total_t if total_t > 0 else 0
                  print('      ' + ttype + ' : ' + str(cnt) + ' ({: .2f}%}'.format(pct))
    print('      asymetrie=' + '{: .3f}'.format(asy))

print('')
if blob_results:
    n_tetra_blob_mean = np.mean([r['final']['n_tetra_final_incremental'] for r in
    n_tri_blob_mean = np.mean([r['final']['n_tri_final'] for r in blob_results])
    r3 = n_tetra_blob_mean / n_tri_blob_mean if n_tri_blob_mean > 0 else 0
    print(' Blob moyen : n_tri={: .0f}'.format(n_tri_blob_mean))

```

```

        + ' n_tetra={:.0f}'.format(n_tetra_blob_mean)
        + ' r3=n_tetra/n_tri={:.4f}'.format(r3))
if null_results:
    n_tetra_null = null_results[0]['final']['n_tetra_final_incremental']
    n_tri_null = null_results[0]['final']['n_tri_final']
    print(' Null : n_tri=' + str(n_tri_null) + ' n_tetra=' + str(n_tetra_null))

print('')
print('=' * 75)
print(' FIN PROTOCOLE F v2')
print('=' * 75)

```